MOST IMPORTANT OOPS QUESTIONS

1. What is the difference between OOP and SOP?

- Include: Definitions, key features of OOP (like encapsulation, inheritance), and procedural programming concepts (functions, linear execution).

2. What is OOP?

- Include: Definition, key principles (like encapsulation, inheritance, polymorphism), and advantages over SOP.

3. Why use OOP?

- Include: Clarity, code reusability, encapsulation, data hiding, problem-solving efficiency, and flexibility through polymorphism.

4. What are the main features of OOP?

- Include: Definitions of inheritance, encapsulation, polymorphism, data abstraction, classes, and objects, along with examples for each.

5. What is an object?

- Include: Definition, attributes and behaviors, real-world examples (e.g., car, dog).

6. What is a class?

- Include: Definition, relationship to objects, and examples.

7. What is the difference between a class and a structure?

- Include: Definitions, usage, characteristics of classes (methods, encapsulation) versus structures (data types).

8. Can you call the base class method without creating an instance?

- Include: Explanation of static methods and inheritance with examples.

9. What is the difference between a class and an object?

- Include: Definitions, relationship (blueprint vs. instance), and examples.

10. What is inheritance?

- Include: Definition, types of inheritance, and examples illustrating its use.

11. What are the different types of inheritance?

- Include: Definitions and examples of single, multiple, multilevel, hierarchical, and hybrid inheritance.

12. What is the difference between multiple and multilevel inheritances?

- Include: Definitions, characteristics, and examples.

13. What is hybrid inheritance?

- Include: Definition, characteristics, and real-world examples.

14. What is hierarchical inheritance?

- Include: Definition, examples, and benefits.

15. What are the limitations of inheritance?

- Include: Challenges, potential coupling issues, and examples of where inheritance may complicate design.

16. What is a superclass?

- Include: Definition, explanation of parent-child relationships, and examples.

17. What is a subclass?

- Include: Definition, relationship with superclass, and examples.

18. What is polymorphism?

- Include: Definition, real-world examples, and how it provides flexibility in programming.

19. What is static polymorphism?

- Include: Definition, characteristics, and example (like method overloading).

20. What is dynamic polymorphism?

- Include: Definition, characteristics, and example (like method overriding).

21. What is method overloading?

- Include: Definition, advantages, and examples of overloaded methods.

22. What is method overriding?

- Include: Definition, scenarios for use, and examples illustrating overriding.

23. What is operator overloading?

- Include: Definition, use cases, and examples with code snippets.

24. Differentiate between overloading and overriding.

- Include: Definitions, scenarios for each, and example code snippets.

25. What is encapsulation?

- Include: Definition, benefits, and examples illustrating data binding.

26. What are ‘access specifiers’?

- Include: Definitions, examples of public, private, and protected access levels.

27. What is the difference between public, private, and protected access modifiers?

- Include: Definitions, usage scenarios, and examples.

28. What is data abstraction?

- Include: Definition, significance, and examples demonstrating abstraction in programming.

29. How to achieve data abstraction?

- Include: Methods like abstract classes and interfaces with examples.

30. What is an abstract class?

- Include: Definition, characteristics, and examples.

31. Can you create an instance of an abstract class?

- Include: Explanation with examples illustrating the inability to instantiate abstract classes.

32. What is an interface?

- Include: Definition, characteristics, and examples showing implementation in classes.

33. Differentiate between data abstraction and encapsulation.

- Include: Definitions, examples, and key differences.

34. What are virtual functions?

- Include: Definition, usage in polymorphism, and examples.

35. What are pure virtual functions?

- Include: Definition, examples, and how they differ from regular virtual functions.

36. What is a constructor?

- Include: Definition, types (default, parameterized), and examples.

37. What is a destructor?

- Include: Definition, purpose, and examples.

38. Types of constructors.

- Include: Definitions and examples of each type of constructor.

39. What is a copy constructor?

- Include: Definition, purpose, and examples.

40. What is the use of ‘finalize’?

- Include: Definition and examples demonstrating its utility in resource management.

41. What is Garbage Collection (GC)?

- Include: Definition, process explanation, and importance in memory management.

42. Differentiate between a class and a method.

- Include: Definitions, roles in OOP, and examples.

43. Differentiate between an abstract class and an interface.

- Include: Definitions, key differences, and examples.

44. What is a final variable?

- Include: Definition, characteristics, and examples.

45. What is an exception?

- Include: Definition, types of exceptions, and examples of scenarios that raise exceptions.

46. What is exception handling?

- Include: Definition, importance, and examples demonstrating exception handling.

47. What is the difference between an error and an exception?

- Include: Definitions, examples, and key differences.

48. What is a try/catch block?

- Include: Definition, structure, and examples demonstrating its use.

49. What is a finally block?

- Include: Definition, use cases, and examples demonstrating the importance of the finally block.

50. What are the limitations of OOPs?

- Include: Potential pitfalls, complexity, performance issues, and examples.

Additional Questions

51. What is a singleton class?
- Include: Definition, purpose, examples, and scenarios for use.
52. What is composition in OOP?
- Include: Definition, differences from inheritance, and examples illustrating composition.

53. What is the SOLID principle?
- Include:Explanationofeachprinciple(SingleResponsibility,Open/Closed,Liskov Substitution, Interface Segregation, Dependency Inversion) and examples.

54. What is the importance of design patterns in OOP?
- Include:Definitionofdesignpatterns,examplesofcommonpatterns(e.g.,Singleton, Factory), and their benefits.

55. What are static and instance methods?
- Include:Definitions,differences,andexamplesdemonstratingusagescenarios.

MASSIVE SUCCESS RATE

- **In-depth Technical Mock**
  - **Crack coding challenges with real Accenture experts.**
- **HR & Managerial Prep**
  - **Master behavioral questions and impress TCS.**
- **Full Interview Simulation**
  - **Ace both technical and HR in one session.**
- **Resume Review**
  - **Identify and fix weaknesses for a standout CV.**
- **Personalized Feedback & Expert Guidance**
  - **Tailored improvement tips to boost success.**

www.primecoding.in